

# 相识北洋（Meeting in Beiyang）局域网社交系统全景使用与开发手册

组长：田华阳

组员：黄嘉俊 康猛 杨浚哲

2026 年 6 月 29 日

## 目录

|          |                           |          |
|----------|---------------------------|----------|
| <b>1</b> | <b>引言与系统概述</b>            | <b>2</b> |
| 1.1      | 课程挑战要求与项目完成情况             | 2        |
| 1.1.1    | 课程终极挑战总体要求                | 2        |
| 1.1.2    | 挑战 1：程序分享                 | 2        |
| 1.1.3    | 挑战 2：文件速传                 | 2        |
| 1.1.4    | 挑战 3：相识北洋                 | 3        |
| <b>2</b> | <b>GitHub 仓库获取与安全更新指南</b> | <b>3</b> |
| 2.1      | GitHub 仓库地址               | 3        |
| 2.2      | 首次克隆与常规更新                 | 3        |
| 2.2.1    | 首次下载                      | 3        |
| 2.2.2    | 常规代码更新                    | 3        |
| 2.3      | 处理代码更新冲突                  | 3        |
| 2.4      | 无 Git 环境下的 ZIP 安全更新步骤     | 4        |
| 2.5      | Git 协作与 Commit 规范         | 4        |
| <b>3</b> | <b>项目结构与模块职责</b>          | <b>4</b> |
| 3.1      | 整体项目目录说明                  | 4        |
| 3.2      | 系统分层设计原则                  | 5        |
| <b>4</b> | <b>功能特性与操作指南</b>          | <b>5</b> |
| 4.1      | 附近的用户与好友请求机制              | 5        |
| 4.2      | 即时通信（Chat）与历史记录           | 5        |
| 4.3      | 文件传输、校验与断点续传              | 6        |
| 4.4      | 北洋空间动态分享墙（Moments）        | 6        |
| <b>5</b> | <b>核心技术设计与架构实现</b>        | <b>6</b> |
| 5.1      | 基于 UDP 的设备发现协议            | 6        |
| 5.2      | 基于 TCP 的点对点通信与协议帧设计       | 6        |
| 5.3      | 断点续传与安全文件传输设计             | 7        |

|                                        |   |
|----------------------------------------|---|
| 目录                                     | 2 |
| 5.4 离线消息与多跳网络中继协议                      | 7 |
| 5.5 SQLite 数据库架构设计                     | 7 |
| 5.6 线程安全与数据库排他锁                        | 7 |
| 5.7 自动保存草稿机制 (Auto-save Draft Pattern) | 8 |
| 6 桌面端安装与快速部署                           | 8 |
| 6.1 基础 Python 环境部署                     | 8 |
| 6.2 Windows 双实例与仿真用户压力测试               | 8 |
| 7 Android 平台编译与移动端适配                   | 8 |
| 7.1 适配设计与兼容性考量                         | 8 |
| 7.2 容器化无缓存 APK 编译命令操作                  | 8 |
| 8 开发者指南与测试运行                           | 9 |
| 8.1 Flet 0.85.3 框架接口规范                 | 9 |
| 8.2 运行本地单元测试                           | 9 |

## 1 引言与系统概述

“相识北洋（Meeting in Beiyang）”是一款基于 Python 3 与 Flet 0.85.3 框架开发的校园局域网点对点（P2P）社交应用程序。在无中心化服务器的物理局域网（LAN）环境下，该应用利用网络广播与异步套接字通信技术，构建起了一套兼具高并发、自动容错与跨平台能力的局域网社交生态体系。

本手册旨在对系统的技术架构、底层协议、环境安装、编译部署（含 Android 平台容器化打包）、测试模拟及各功能模块的操作细节进行全面深入的拆解与指导，为用户和开发者提供完整的项目指南。

### 1.1 课程挑战要求与项目完成情况

本项目针对课程提出的三项核心挑战进行了完整的设计与实现。以下是各阶段挑战的具体要求与本项目在“相识北洋”系统中的技术实现对照情况：

#### 1.1.1 课程终极挑战总体要求

本课程终极挑战要求开发包含 3 个挑战的跨平台局域网社交软件，提供源码、APK 安装包与项目说明书。本项目在 `challenge3` 文件夹中对上述要求进行了全功能整合：

- **功能全集成：**在“相识北洋”这一款软件中完美整合了剪贴板分享、大文件断点续传、好友匹配与离线 Gossip 中继等所有核心功能。
- **多平台打包：**编写了防缓存的 Android 自动化编译脚本 `build-apk-clean.ps1`，用于生成最终的 `.apk` 安装包。
- **手册撰写：**撰写了本 LaTeX 技术手册并编译成对应的 PDF 版本。

#### 1.1.2 挑战 1：程序分享

挑战 1 侧重于两端设备间的代码/文本的快速分享与剪贴板集成。本项目的完成情况如下：

- **剪贴板完美集成：**基于 Flet 的底层机制，在聊天详情页中深度集成了系统剪贴板。消息详情和文件传输卡片中均配备了一键复制文本和文件路径（复制路径）的逻辑。
- **局域网设备自动发现：**设计了 `UDPService` 服务（监听端口 8890），各端在局域网内定时广播 PING 帧宣告上线，并动态构建局域网节点映射表。
- **跨平台连接：**支持 Windows 桌面端和 Android 移动端的面对面直连通信。

#### 1.1.3 挑战 2：文件速传

挑战 2 侧重于大文件的传输进度显示、中断重传、压缩包解压与安全哈希校验。本项目的完成情况如下：

- **自定义保存目录：**集成了 Flet `FilePicker` 交互，并在设置中允许用户自主配置默认的文件接收目录。
- **ZIP 自动解压：**检测到接收到的文件是 ZIP 压缩包时，消息卡片提供“解压 ZIP”一键后台异步解压缩。
- **分块传输与进度展示：**消息服务将文件切分为 256KB 块进行传输，前端 UI 实时渲染传输进度条、传送速度以及取消动作。

- **断点续传与完整性校验：**传输中断后可基于 part 文件的实际尺寸进行断点续传；传输完成后在 FileStore 中利用 SHA-256 哈希值进行整机安全校验。

#### 1.1.4 挑战 3：相识北洋

挑战 3 侧重于个人简介条件匹配、好友管理、多跳离线消息中继以及移动状态下对 IP 变更的动态适应。本项目的完成情况如下：

- **自动审批匹配：**用户可配置兴趣标签与匹配条件，接收申请时自动判定交集标签数，符合条件的自动同意加为好友。
- **好友管理系统：**实现了分组、好友搜索、删除以及实时聊天记录数据库持久化。
- **多跳离线消息中继：**实现了最大 3 跳的 Gossip 中继机制。当接收方离线时，消息自动通过局域网内活跃的在线共同好友（中继节点）暂存并在后续偶遇时投递。
- **动态 IP 漫游支持：**好友关系建立后，若用户切换网络导致 IP 地址改变，可通过后台 UDP 持续广播发现或心跳包（HEARTBEAT）自动更新本地好友的在线 IP 及端口，维持 TCP 会话的连续性。

## 2 GitHub 仓库获取与安全更新指南

### 2.1 GitHub 仓库地址

项目托管于 GitHub，官方仓库地址为：<https://github.com/Ortenssia/Meeting-in-Beiyang.git>

### 2.2 首次克隆与常规更新

#### 2.2.1 首次下载

使用标准 Git 工具克隆仓库到本地：

```
git clone https://github.com/Ortenssia/Meeting-in-Beiyang.git
cd Meeting-in-Beiyang
```

#### 2.2.2 常规代码更新

当仓库有代码提交时，请在项目根目录下执行以下命令拉取最新代码：

```
git pull origin main
```

### 2.3 处理代码更新冲突

如果您在本地对某些代码进行了临时修改，导致 git pull 时提示冲突并失败，请执行以下命令强制覆盖本地代码（这只会覆盖代码，您的本地聊天记录和好友数据库不会丢失）：

```
git fetch --all
git reset --hard origin/main
```

## 2.4 无 Git 环境下的 ZIP 安全更新步骤

如果您是直接从 GitHub 网页下载的 ZIP 压缩包，后续更新时请千万不要直接用新解压的整个文件夹覆盖原有文件夹。这会导致您的好友关系和历史消息被彻底抹除。请按照以下步骤进行安全覆盖：

1. 解压最新下载的 ZIP 压缩包；
2. 将解压出来的 core/ 目录以及根目录下的 requirements.txt、pyproject.toml 等代码文件复制并覆盖到您原有的本地目录中；
3. 千万不要覆盖或删除原有本地目录中的 .runtime/ 文件夹以及 assets/data/friends.db 文件。这两个路径是本地数据库和身份信息的存放地。

## 2.5 Git 协作与 Commit 规范

所有开发人员在提交代码时应遵循以下规范：

- 目标主分支为 main 分支。
- 所有的 Commit 提交信息末尾，必须追加 Co-Authored 署名，格式如下：

```
Fix socket connection issue in message relaying.
```

```
Co-Authored-By: Claude <noreply@anthropic.com>
```

# 3 项目结构与模块职责

## 3.1 整体项目目录说明

项目的核心文件和代码包按如下模块化组织：

```
core/ # 核心源代码根目录
config/ # 全局路径配置及跨平台路径适配
  paths.py # SQLite、缓存、文件接收目录的动态定位器
frontend/ # 前端界面逻辑
  views/ # 核心视图页面
    discover.py # 附近发现与好友请求页面
    friends.py # 好友列表与管理页面
    chat.py # 即时聊天与分块文件传输页面
    moments.py # P2P 朋友圈/北洋空间动态分享墙
    profile.py # 个人信息及自动保存设置页面
  app.py # BeiyangApp 控制器，前端逻辑与后端服务的纽带
  theme.py # 统一的 Material 3 配色方案与设计系统
backend/ # 后端核心服务
  services/ # 独立运行的网络与数据层服务
    social_runtime.py # 系统生命周期总控、回调挂载与协调中心
    friend_db.py # SQLite 数据库核心（包含好友、聊天记录、中继消息）
    udp_service.py # 基于 UDP 的局域网主动扫描与被动发现服务
    connection_manager.py # 基于 TCP Socket 连结池的底层双向通信与心跳管理
    message_service.py # 消息中继机制、好友同步与通信路由解析
    file_transfer_state.py # 传输任务状态监控器（断点状态、速度、进度）
```

```
file_store.py # 文件接收安全写控制与哈希完整性校验
update_service.py # 模拟软件版本更新中继发布服务
shared/ # 共享常量与网络协议包
protocol.py # 定义二进制协议头、握手和帧格式
file_message.py # 前后端共享传输状态数据帧结构
assets/ # UI 静态资源（系统字体、内置头像、ICO 图标）
docs/ # 系统相关文档目录
operations/ # 运维与仿真测试工具包
run_local_pair.ps1 # 双实例快速联调运行脚本
run_random_users.ps1 # 大规模随机节点稳定性压力测试脚本
build-apk-clean.ps1 # 容器化 Android APK 零缓存编译打包脚本
docker-build-apk.sh # Linux 下的 Docker 编译 Shell 脚本
pyproject.toml # Flet 项目打包依赖和版本配置文件
requirements.txt # 独立部署依赖库清单
```

### 3.2 系统分层设计原则

本项目遵循严格的分层隔离设计：

1. **前端视图层**：不直接操作网络 Socket，不直接连接 SQLite 数据库，所有数据变更均通过 BeiyangApp 提供的业务 API 执行。
2. **后端服务层**：运行于独立线程中，提供高效的底层锁机制以确保数据安全性。各服务状态的变化通过事件回调（Callbacks）传递给应用管理器，进而驱动前端 UI 更新。
3. **共享与配置层**：提供静态协议常量和安全路径定位，屏蔽操作系统（Windows / macOS / Linux / Android）的文件系统 and 底层接口差异。

## 4 功能特性与操作指南

### 4.1 附近的用户与好友请求机制

- **自动扫描**：进入“发现”页面后，应用会通过 UDP 自动扫描并展示当前局域网内的其他活动节点。
- **好友条件审核**：在“个人资料”中，您可以配置兴趣标签并设置“最低匹配标签数”。如果对方发送申请时的兴趣标签与您的匹配数少于此数值，系统会自动拒绝申请。
- **申请处理**：可在发现页的“好友请求”列表中点击“同意”或“拒绝”。支持在设置中开启“自动同意好友申请”开关。

### 4.2 即时通信（Chat）与历史记录

- **单聊与群聊**：双击好友可发起点对点 TCP 聊天。系统同时支持创建局域网多点群聊，群聊元数据会在成员之间自动同步。
- **未读红点与滚动**：当有新消息且当前不处于该会话时，好友头像上会显示未读消息数红点。进入聊天后，窗口会自动平滑滚动到底部。

### 4.3 文件传输、校验与断点续传

- **拖拽与卡片发送：**直接将文件拖入聊天窗口或点击“发送文件”按钮发送。
- **传输控制与断点续传：**大文件传输过程中支持随时“取消”。若传输中断，双击“重试”会自动获取本地已写入的字节位置，发起断点续传。
- **完成操作：**文件传输完成后，卡片上会自动提供“打开文件”（调用系统默认应用）、“定位目录”（在资源管理器中选中并高亮该文件）、“复制路径”按钮。如果是 ZIP 压缩包，还会提供“解压 ZIP”按钮。

### 4.4 北洋空间动态分享墙（Moments）

- **动态发布：**进入“北洋空间”页面，您可以输入文字，并点击图片按钮添加一张本地图片作为附件，点击“发布动态”发布。
- **P2P 动态同步：**空间动态不依赖服务器，发布后将暂时保存在本地数据库中。当您与任何好友建立 TCP 连接时，系统会自动在后台将您以及您收集到的最新动态同步给对端。
- **动态互动：**您可以为好友的动态点赞或发表评论，这些互动信息同样会通过 P2P 链条同步给所有共同好友。

## 5 核心技术设计与架构实现

### 5.1 基于 UDP 的设备发现协议

用户启动应用后，UDP 服务（监听端口 8890）将自动启动并广播包含以下载荷的 JSON 帧：

```
{
  "name": "用户昵称",
  "port": 7779,
  "action": "ping"
}
```

发现机制流程：

- **周期性广播：**每隔 5 秒向局域网广播子网段发送一次 ping 帧。
- **主动扫描响应：**当收到新用户的 ping 帧时，本端会主动回应一个单播 pong 帧。双方获得对方的 IP 地址和 TCP 端口号，将对方列入“附近的用户”。

### 5.2 基于 TCP 的点对点通信与协议帧设计

两个已经加为好友的节点在进行通信前，将通过 TCP Socket 进行三次握手式状态验证，协议传输通过自定义二级帧结构解析：

1. **版本握手：**建立连接后立即交换彼此的 `pyproject.toml` 中的软件版本 and UUID。
2. **心跳维持（Heartbeat）：**连接保持期间，每隔 15 秒发送一次心跳空包以检测对端死活。若连续 3 次无回应则断开连接。

### 5.3 断点续传与安全文件传输设计

文件分块传输（每块大小默认 256KB）的协议状态流程如图所示：

- **握手阶段：**发送方提供文件全名、字节长度 and SHA-256 哈希值。接收方检索数据库，若发现有未传输完毕的旧任务记录，则返回当前已写入文件指针位置（例如 `Offset: 4194304` 字节），自动激活断点续传。
- **传输阶段：**发送方从指定 `Offset` 开始读取文件块并发送，接收方以追加模式写入目标路径文件。
- **校验阶段：**传输结束，接收方独立重新计算整机文件的 SHA-256 并同原始哈希进行匹配。若匹配成功，状态置为已完成，否则丢弃文件报错。

### 5.4 离线消息与多跳网络中继协议

对于暂时离线（UDP 未在线且 TCP 断开）的好友，消息发送将遵循有限跳数的八卦广播协议（Gossip-based Store-and-Forward Routing）：

- 本地将该消息标记为“离线”状态并保存在本地数据库表 `pending_messages` 中。
- 当检测到局域网中存在任意与本端和目标端同为好友的“在线第三方”时，本端会尝试将此消息包裹为 `RELAY_MESSAGE` 并加密传输给中继节点。
- 中继节点作为“消息中继器”将此消息持久化到本地库中，最大中继跳数控制为 3 跳，在中继节点偶遇目标节点上线时最终将消息路由送达。

### 5.5 SQLite 数据库架构设计

所有本地的友好、关系、设置和消息记录均保存在本地 SQLite 数据库中。数据库包含以下核心表结构：

- `friends`: 保存好友的 UUID、昵称、最近在线 IP、端口、头像名、个人简介及状态。
- `chat_history`: 聊天消息记录表，包含消息唯一标识 `msg_id`、发送方、接收方、消息文本、时间戳和状态。
- `pending_messages`: 离线缓存消息表。
- `received_msg_ids`: 中继消息去重表，用于过滤已成功接收的中继消息，防止洪泛风暴。
- `groups & group_chat_history`: 本地群聊及群聊历史表。
- `moments & moment_comments`: 北洋空间动态及互动评论表。

### 5.6 线程安全与数据库排他锁

由于界面渲染线程、TCP 接收线程、UDP 发现线程都会并发操作 SQLite，为了避免 SQLite 发生锁死错误，`friend_db.py` 实现了严格的线程排他锁机制。所有写操作必须加锁：

```
def save_chat_message(self, from_name, to_name, content, timestamp, msg_id):
    with self._lock:
        cursor = self.conn.cursor()
        cursor.execute("INSERT INTO chat_history ...")
        self.conn.commit()
```



## 5.7 自动保存草稿机制 (Auto-save Draft Pattern)

为了避免移动端和桌面端用户频繁点击保存按钮，个人资料与设置页的输入框采用了 Auto-save 机制。在输入过程中，`on_change` 回调会持续将输入值同步到内存中的草稿变量 `_draft_*`。当输入框失去焦点 (`on_blur`) 或用户点击输入框外部 (`on_tap_outside`) 时，触发自动写入数据库。**设计考量：**由于部分平台上 `on_blur` 可能会先于最后一次 `on_change` 完成触发，脚本在保存时优先读取内存草稿 `_draft_*`，其次回退读取控件值，彻底规避了时序竞态问题。

# 6 桌面端安装与快速部署

## 6.1 基础 Python 环境部署

推荐使用 Python 3.10.x - 3.12.x 环境运行：

```
# 创建并激活环境
python -m venv .venv
.\.venv\Scripts\Activate.ps1
# 安装项目依赖
pip install -r requirements.txt
```

## 6.2 Windows 双实例与仿真用户压力测试

项目提供了完善的本端快速联调仿真脚本：

1. 快速双实例联调：在 PowerShell 下运行 `operations/run_local_pair.ps1`，该脚本会自动拉起 Alice 和 Bob 两个配置隔离的窗口进行收发测试。
2. 大规模网络仿真：运行 `run_random_users.ps1`：

```
# 模拟启动 10 个后台隐藏节点（端口自动递增分配）以测试局域网发现和中继消息路由压力
powershell -File operations/run_random_users.ps1 -Count 10 -NoWindow $true
```

测试完毕后可使用以下命令一键清理：

```
Get-Process python | Stop-Process
```

# 7 Android 平台编译与移动端适配

## 7.1 适配设计与兼容性考量

Android 作为受限移动系统，相比桌面端在网络与文件系统有极大的限制。本项目在 `core` 中预设了针对 Android 的一系列底层机制：

## 7.2 容器化无缓存 APK 编译命令操作

打包脚本 `build-apk-clean.ps1` 已经被重构，彻底移除了可能导致旧代码复用的 Gradle 全局缓存机制。

要生成无缓存、最新版本的 APK，请执行以下命令：

表 1: 移动端 Android 兼容性与解决方案表

| 限制分类  | Android 底层表现                       | 解决方案                                          |
|-------|------------------------------------|-----------------------------------------------|
| 网络通信  | 锁屏/后台时 UDP 广播无法收发                  | 使用 Jnius 调取系统 MulticastLock 服务                |
| 文件系统  | assets 目录只读，无法修改配置文件               | 自动将可写数据迁移到 app 专属私有存储目录                       |
| UI 接口 | 移动端无法导入 tkinter 模块                 | 捕获异常，使用原生网页 FilePicker 替代                     |
| 交互限制  | 无桌面级的 <code>os.startfile</code> 功能 | 通过 <code>am start</code> 调用 Android Intent 广播 |
| 资源文件  | 无法解析 ICO 格式图标                      | 增加平台判断，加载 PNG 适应系统图标                          |

```
# 运行全新的防缓存编译
.\build-apk-clean.ps1
```

编译输出将位于项目根目录的 `build/apk/meeting-in-beiyang.apk` 下。

## 8 开发者指南与测试运行

### 8.1 Flet 0.85.3 框架接口规范

为了防止编写出因 Flet 版本升级导致不兼容的代码，所有开发人员需严格遵守以下控件参数定义：

- **TextField:** 使用 `helper="..."` 来展示输入框下方的辅助提示文本（禁止使用旧版的 `helper_text`）。
- **Dropdown:** 下拉选择事件必须绑定在 `on_select` 事件上（禁止绑定在 `on_change` 上）。
- **Switch:** 状态切换事件保持绑定在 `on_change` 上。
- **FilePicker:** 在 Flet 0.85+ 中，FilePicker 属于 Service，必须添加到 `page.services.append(picker)` 中，不可再作为 Overlay 控件直接加入 `page.overlay`。

### 8.2 运行本地单元测试

在提交新的代码或分支前，请利用本地 Pytest 框架对数据库和中继机制进行一致性测试：

```
# 运行测试套件
python -m pytest core/tests -q
```